



Les Chroniques de Valandor



Spécifications Techniques & Conception du Projet

Projet	Les Chroniques de Valandor — RPG Textuel
Langage	Java 17+
Type	Programmation Orientée Objet (POO)
Niveau	Intermédiaire — toutes fonctionnalités
Document	Spécifications, Entités & Choix techniques

1. Vision du Projet

Les Chroniques de Valandor est un jeu de rôle entièrement textuel où le joueur incarne un aventurier explorant un monde médiéval-fantastique. L'objectif pédagogique central est de maîtriser la programmation orientée objet à travers une application concrète, ludique et évaluée.

1.1 Objectifs pédagogiques

- **Héritage & abstraction** : hiérarchies LivingEntity → Player / Enemy / NPC
- **Composition** : Player possède un Inventory, des EquipmentSlots, un QuestLog
- **Patron de conception** : Command Pattern pour l'interprétation des actions
- **Sérialisation** : sauvegarde/chargement JSON via Gson
- **Architecture logicielle** : séparation claire des responsabilités entre classes

1.2 Scénario

Dans les terres anciennes de Valandor, une menace obscure s'éveille. Le joueur lit des descriptions, entre des commandes texte (ex : "attaquer goblin", "aller nord", "utiliser potion") et interagit avec un monde vivant. Les combats utilisent un système de dés (D6, D20) pour introduire aléatoire et stratégie.

2. Spécifications Fonctionnelles

Fonctionnalité	Description	Priorité
Exploration	Navigation entre lieux connectés (N/S/E/O), descriptions, objets au sol, PNJ présents	Haute
Inventaire	Ajouter / retirer / consulter objets, limite de poids, affichage formaté	Haute
Équipement	Emplacements arme / armure / casque, bonus automatiques sur stats	Haute
Combat	Tour par tour, lancer D20 (toucher) + dés dégâts, bonus équipement, butin à la mort	Haute
Progression	XP → niveaux, courbe progressive, amélioration stats à chaque montée	Haute
Quêtes	Accepter/suivre/compléter des quêtes, conditions diverses, récompenses	Moyenne
PNJ & dialogues	Dialogues textuels, propositions de quêtes ou de services	Moyenne
Sauvegarde	Sérialisation JSON complète (position, stats, inventaire, quêtes), chargement	Haute
Parsing commandes	Interprétation texte, dictionnaire de commandes, aliases, gestion d'erreurs	Haute
Aide contextuelle	Commande "aide" affichant les actions disponibles selon le contexte	Basse

3. Entités du Jeu

L'architecture repose sur deux grandes hiérarchies : les entités vivantes et les objets. Chaque classe a une responsabilité unique et clairement définie.

3.1 Hiérarchie des Entités Vivantes

LivingEntity «Classe abstraite»	
Classe mère de toutes les entités vivantes. Factorise les attributs communs (PV, stats de combat) et les comportements partagés. Ne peut pas être instanciée directement.	
Attributs	Méthodes principales
name : String	takeDamage(int amount)
maxHp : int	heal(int amount)
currentHp : int	isAlive() : boolean
attackPower : int	attack(LivingEntity target)
defense : int	getStats() : String
isAlive : boolean	– abstract –

Player

«extends LivingEntity»

Le personnage contrôlé par le joueur. Gère l'expérience, les niveaux, l'inventaire et l'équipement via composition. C'est le pivot central du jeu.

Attributs	Méthodes principales
xp : int	gainXP(int xp)
level : int	levelUp()
inventory : Inventory	equipItem(Equipment item)
equipment : EquipmentSlots	unequipItem(String slot)
questLog : QuestLog	useItem(Item item)
currentLocation : Location	move(String direction)
	saveToJson() : JsonObject
	loadFromJson(JsonObject)

Enemy

«extends LivingEntity»

Adversaire combattable. Offre de l'XP et du butin à sa mort. Peut avoir des comportements spécifiques selon son type (Goblin, Dragon, etc.).

Attributs	Méthodes principales
xpReward : int	dropLoot() : List<Item>
loot : List<Item>	getXPReward() : int
lootChance : float	takeTurn(Player player)
description : String	toJson() : JsonObject

NPC

«extends LivingEntity»

Personnage non-joueur avec lequel le joueur peut dialoguer, obtenir des quêtes ou commercer. N'est pas combattable directement.

Attributs	Méthodes principales
dialogue : Map<String, String>	talk(String topic) : String
availableQuests : List<Quest>	offerQuest(Player player)
shopInventory : List<Item>	trade(Player player)
	toJson() : JsonObject

3.2 Hiérarchie des Objets

Item

«Classe abstraite»

Classe mère de tous les objets du jeu. Définit les propriétés communes et force chaque sous-classe à implémenter la méthode use().

Attributs	Méthodes principales
-----------	----------------------

name : String	use(Player player)
description : String	toJson() : JsonObject
weight : float	fromJson(JsonObject) : Item
value : int	toString() : String
	— abstract —

Weapon

«extends Item»

Arme équipable qui augmente les dégâts d'attaque. Le calcul de dégâts utilise les dés configurés (ex : 2D6 + bonus).

Attributs	Méthodes principales
damage : int	use(Player player)
damageType : String	getDamageRoll() : int
diceCount : int	toJson() : JsonObject
diceSides : int	

Armor

«extends Item»

Armure équipable sur un emplacement précis (torse, casque). Augmente la défense du personnage lorsqu'elle est portée.

Attributs	Méthodes principales
defenseBonus : int	use(Player player)
slot : String	applyBonus(Player p)
durability : int	removeBonus(Player p)
	toJson() : JsonObject

Consumable

«extends Item»

Objet à usage unique qui disparaît après utilisation. Peut restaurer des PV, booster temporairement les stats, etc.

Attributs	Méthodes principales
effect : String	use(Player player)
effectValue : int	applyEffect(Player p)
duration : int	toJson() : JsonObject

Potion

«extends Consumable»

Cas particulier de Consumable spécialisé dans la restauration de PV. Type de potion variable (soin, mana, antidote).

Attributs	Méthodes principales
healAmount : int	use(Player player)
potionType : String	toJson() : JsonObject

3.3 Entités du Monde

Location	
«Classe concrète»	
Un lieu du monde de Valandor. Connecté à d'autres lieux par des sorties directionnelles. Peut contenir des objets au sol et des entités (ennemis, PNJ).	
Attributs	Méthodes principales
name : String	addExit(String dir, Location loc)
description : String	getExit(String dir) : Location
exits : Map<String, Location>	addItem(Item item)
items : List<Item>	removeItem(Item item)
entities : List<LivingEntity>	describe() : String
	toJson() : JsonObject

Quest	
«Classe concrète»	
Quête que le joueur peut accepter et accomplir. Définit des objectifs à remplir (tuer X ennemis, trouver un objet, visiter un lieu) et offre des récompenses à la complétion.	
Attributs	Méthodes principales
title : String	checkCompletion(Player p) : boolean
description : String	complete(Player p)
objectives : List<Objective>	addObjective(Objective o)
rewards : List<Item>	getStatus() : String
xpReward : int	toJson() : JsonObject
isCompleted : boolean	

3.4 Classes de Support

Inventory	
«Classe concrète (composition)»	
Gestion de l'inventaire du joueur par composition. Contrôle la limite de poids et expose les opérations CRUD sur les objets possédés.	
Attributs	Méthodes principales
items : List<Item>	addItem(Item item) : boolean
maxWeight : float	removeItem(Item item)
currentWeight : float	getItems() : List<Item>
	hasItem(String name) : boolean
	display() : String
	toJson() : JsonObject

Dice	
«Classe utilitaire»	

Encapsule la génération de nombres aléatoires simulant des lancers de dés. Centralisée pour faciliter les tests (injection possible d'un Random mocké).

Attributs	Méthodes principales
<code>random : Random</code>	<code>roll(int sides) : int</code>
	<code>rollMultiple(int count, int sides) : int</code>
	<code>rollD6() : int</code>
	<code>rollD20() : int</code>

CommandParser

«Classe concrète»

Interprète les entrées texte du joueur. Implémente le patron Command en mappant chaque mot-clé à une classe de commande concrète. Gère les aliases et les erreurs de saisie.

Attributs	Méthodes principales
<code>commands : Map<String, Class<?>></code>	<code>parse(String input) : Command</code>
<code>aliases : Map<String, String></code>	<code>registerCommand(String key, Class<?>)</code>
	<code>normalize(String input) : String</code>
	<code>tokenize(String input) : String[]</code>

4. Patron de Conception — Command Pattern

Chaque action du joueur est encapsulée dans une classe dédiée implémentant l'interface Command. Le CommandParser instancie la bonne classe selon l'entrée du joueur.

Command

«Interface»

Contrat minimal que toute commande doit respecter. Permet d'ajouter de nouvelles commandes sans modifier le code existant (principe Ouvert/Fermé).

Attributs	Méthodes principales
	<code>execute(GameContext context)</code>
	— une seule méthode —

Commandes concrètes implémentées :

Classe	Exemple entrée joueur	Action effectuée
<code>MoveCommand</code>	"aller nord" / "n"	Déplace le joueur vers le lieu lié à la direction
<code>AttackCommand</code>	"attaquer goblin"	Lance un combat tour par tour contre la cible
<code>PickUpCommand</code>	"prendre épée"	Ramasse un objet présent dans le lieu
<code>UseItemCommand</code>	"utiliser potion"	Utilise un objet de l'inventaire
<code>EquipCommand</code>	"équiper épée"	Équipe un objet dans le slot approprié

UnequipCommand	"déséquiper casque"	Retire un équipement du slot
TalkCommand	"parler marchand"	Lance un dialogue avec un PNJ
InventoryCommand	"inventaire"	Affiche le contenu de l'inventaire
QuestCommand	"quêtes"	Affiche le journal de quêtes
SaveCommand	"sauvegarder"	Séréalise l'état du jeu en JSON
LoadCommand	"charger"	Déséréalise et restaure une partie sauvegardée
HelpCommand	"aide"	Affiche les commandes disponibles

5. Choix des Technologies

Composant	Technologie	Justification
Langage	Java 17+	LTS stable, typage fort, POO native, records et sealed classes disponibles
Build tool	Maven	Gestion des dépendances standard, structure de projet claire (src/main, src/test)
CLI / I/O	Scanner (natif)	Aucune dépendance externe nécessaire pour lire l'entrée texte de l'utilisateur
CLI avancée	Picocli (optionnel)	Si parsing d'arguments avancé nécessaire, léger et annoté
Sérialisation JSON	Gson (Google)	Simple, annotations @SerializedName, toJson()/fromJson() suffisants pour le projet
Tests unitaires	JUnit 5	Tests des mécaniques clés : dés, combat, inventaire, parsing
Diagramme UML	PlantUML / Draw.io	Génération ou dessin du diagramme de classes requis dans les livrables
IDE recommandé	IntelliJ IDEA	Support Java excellent, refactoring, génération de code automatique

5.1 Structure Maven du Projet

```
valandor/ |─ src/ | | |─ main/java/com/valandor/ | | |─ entity/ (LivingEntity,
Player, Enemy, NPC) | | |─ item/ (Item, Weapon, Armor, Consumable, Potion) |
| |─ world/ (Location, Quest, Objective) | | |─ command/ (Command
interface + toutes les commandes) | | |─ system/ (Inventory, Dice,
CommandParser, GameContext) | | |─ Main.java | | |─ test/java/com/valandor/ |
|─ DiceTest.java | | |─ InventoryTest.java | | |─ CombatTest.java |─ saves/
(fichiers JSON de sauvegarde) |─ pom.xml |─ README.md |─ AI_USAGE.md |─ uml/diagram.png
```

6. Phases de Développement

Phase	Nom	Contenu
-------	-----	---------

1	Fondations	LivingEntity, Player, Location, déplacement entre lieux, affichage des descriptions
2	Objets & Inventaire	Item, Weapon, Armor, Potion, Inventory, pickup, use, équipement
3	Combat	Dice, système tour par tour, Enemy, calcul dégâts/défense, butin, mort
4	Progression	XP, montée de niveau, courbe équilibrée, amélioration stats
5	Quêtes & PNJ	Quest, Objective, NPC, dialogues, récompenses, QuestLog
6	Sauvegarde	Sérialisation Gson complète, toJson/fromJson sur toutes entités, Save/Load
7	Finitions	Aide contextuelle, gestion des erreurs, corrections de bugs, tests JUnit

7. Livrables

7.1 Livrables attendus

- **Code source complet** — organisé selon la structure Maven ci-dessus
- **README.md** — guide d'utilisation, commandes disponibles, prérequis
- **Diagramme UML** — héritage, composition, associations entre classes
- **Vidéo de démonstration** — 5 à 10 minutes montrant toutes les fonctionnalités
- **AI_USAGE.md** — documentation de chaque utilisation d'IA (outil, prompt, résultat, adaptation)